

ATIVIDADE REMOTA — MÁQUINAS DE TURING

Teoria da Computação

Estudo, simulação e reflexão sobre os limites computacionais

Disciplina

Teoria da Computação

Tema

Máquinas de Turing

Modalidade

Remota

Valor

1,0 ponto



Material obrigatório

Assista ao vídeo antes de realizar a atividade

Para realizar esta atividade, assista ao vídeo:

Akitando #86 — O Computador de Turing e Von Neumann: Por que calculadoras não são computadores?

Orientação: Assista ao material com atenção e utilize os conceitos apresentados para responder às questões e desenvolver a atividade de simulação.

1. Objetivos da atividade

Ao final da atividade, o estudante deverá ser capaz de:

- Compreender o conceito de Máquina de Turing.
- Identificar os principais componentes de uma Máquina de Turing.
- Compreender a importância das Máquinas de Turing para a computação.
- Construir e simular uma Máquina de Turing simples.
- Reconhecer que existem problemas que não podem ser resolvidos por algoritmos, compreendendo os limites da computação.

2. Conteúdo

- Conceito de Máquina de Turing.
- Fita, cabeça de leitura/escrita e estados.
- Alfabeto e regras de transição.
- Funcionamento de uma Máquina de Turing.
- Simulação computacional.
- Computabilidade e limites computacionais.

3. Desenvolvimento da atividade

Etapa 1 — Introdução

Após assistir ao vídeo e estudar o material disponibilizado, responda às questões abaixo.

1. O que é uma Máquina de Turing?

É um computador "de papel" que o Alan Turing inventou em 1936. Ela tem uma fita infinita dividida em quadradinhos, uma cabeça que lê e escreve um símbolo de cada vez e anda para os lados, e uma lista de regras que diz o que fazer em cada situação. Mesmo sendo simples assim, ela consegue fazer qualquer cálculo ou algoritmo que um computador de verdade faz, por isso é usada para definir o que é computar.

2. Quais são os principais componentes de uma Máquina de Turing?

- Fita: onde fica a palavra de entrada e onde a máquina vai anotando as coisas.
- Cabeça de leitura/escrita: lê o quadradinho em que está, escreve outro símbolo e anda 1 casa para a esquerda ou direita.
- Estados: a "situação" em que a máquina está (ex.: procurando um 1, voltando...).
- Alfabeto: os símbolos que podem aparecer na fita.
- Regras de transição: "se estou no estado X e li Y, escrevo Z, ando para tal lado e vou para o estado W".

3. Qual é a importância das Máquinas de Turing para a computação?

Antes dela não existia uma definição certinha do que é um algoritmo. Com a máquina deu para provar o que um computador consegue e o que ele nunca vai conseguir fazer. Ela também é a ideia por trás do computador moderno: o Von Neumann usou ela para criar a arquitetura em que programa e dados ficam juntos na memória, que é como os computadores funcionam até hoje. Quando dizem que uma linguagem é "Turing completa", quer dizer que ela faz tudo que a máquina faz.

4. Qual é a relação entre Máquina de Turing e algoritmo?

Algoritmo é uma receita de passos para resolver um problema, e a Máquina de Turing é o jeito "oficial" de escrever essa receita: cada regra da tabela é um passo. A ideia (tese de Church-Turing) é que se dá para resolver com algoritmo, dá para resolver com uma Máquina de Turing. E o contrário também: se nenhuma Máquina de Turing resolve, não tem algoritmo que resolva.

Etapa 2 — Simulação

Utilize um software ou simulador de Máquina de Turing indicado pelo professor.

Crie uma máquina capaz de reconhecer palavras da forma:

$$0^n 1^n$$

Exemplos aceitos

- 01
- 0011
- 000111
- 00001111

Exemplos rejeitados

- 0
- 1
- 001
- 011
- 00111

Desafio: A máquina deverá verificar se existe a mesma quantidade de símbolos **0** e **1**, seguindo a lógica de funcionamento de uma Máquina de Turing.

Etapa 3 — Registro da simulação

Após executar a máquina, registre:

- A palavra utilizada como entrada.
- Os estados percorridos.
- O resultado da execução: **ACEITA** ou **REJEITA**.
- Uma captura de tela da simulação.

Realize pelo menos **3 testes**:

- 2 entradas que devem ser aceitas.
- 1 entrada que deve ser rejeitada.

Registro dos testes

Teste	Entrada	Resultado esperado	Resultado obtido	Estados percorridos
1	01	ACEITA	ACEITA	início → procura_1 → volta → início → confere → aceita
2	00001111	ACEITA	ACEITA	(início → procura_1 → volta) 4 vezes, depois início → confere → aceita
3	011	REJEITA	REJEITA	início → procura_1 → volta → início → confere (sobrou um 1, sem regra)

Descrição da Máquina de Turing criada:

Minha máquina marca um 0 e um 1 de cada vez. No estado início ela troca o primeiro 0 por A e vai para procura_1, que anda para a direita até achar um 1 e troca por B. O estado volta leva a cabeça de volta até o A e tudo recomeça. Quando o início acha um B no lugar de um 0, os 0 acabaram, e o estado confere olha se até o fim da fita só tem B. Se tiver, ACEITA. Se sobrar algum 0 ou 1 sem par, não tem regra e ela REJEITA. A tabela de regras e os prints estão no anexo, no fim do arquivo (páginas 9 a 12).

Etapa 4 — Reflexão sobre os limites computacionais

Uma Máquina de Turing consegue resolver qualquer problema? Explique com suas palavras por que existem problemas que não podem ser resolvidos por algoritmos.

Não. Ela resolve tudo que tem algoritmo, mas existem problemas que não têm algoritmo nenhum. O mais famoso é o problema da parada: descobrir, olhando um programa qualquer, se ele vai terminar ou ficar rodando para sempre. O Turing provou que é impossível fazer um programa que responda isso certo para todos os casos. A ideia é que, se esse programa existisse, daria para montar outro que pergunta sobre ele mesmo e faz o contrário da resposta, e aí a resposta sempre ficaria errada. Então não é questão de ter um computador mais rápido ou com mais memória: mesmo com fita infinita, esses problemas continuam sem solução.

4. Entrega

O estudante deverá enviar **um único arquivo** contendo:

- Respostas da Etapa 1;
- Descrição da Máquina de Turing criada;
- Capturas de tela da simulação;
- Resultados dos 3 testes;
- Resposta da reflexão sobre os limites computacionais.

Formato: PDF ou documento do Word.

5. Critérios de avaliação — 1,0 ponto

Critério	Valor
Compreensão do conceito e importância das Máquinas de Turing	0,20
Construção/configuração da Máquina de Turing	0,30
Realização e registro da simulação	0,20
Interpretação dos resultados	0,10
Compreensão dos limites computacionais	0,20
Total	1,00

6. Questão final

Problema para reflexão:

Imagine que você recebeu um problema computacional muito complexo. Como saber se ele é apenas difícil de resolver ou se, na verdade, não existe nenhum algoritmo capaz de resolvê-lo para todos os casos?

Explique utilizando os conceitos estudados sobre **Máquinas de Turing, computabilidade e limites computacionais**.

Um problema difícil tem solução, só que demora. Por exemplo, testar todas as combinações de uma senha: é demorado, mas uma hora termina e dá a resposta certa. Isso é assunto de complexidade de algoritmos ($O(n^2)$, $O(2^n)$...).

Um problema sem algoritmo (indecidível) é outra coisa: não existe nenhuma Máquina de Turing que pare com a resposta certa para todas as entradas.

Para descobrir qual é o caso: (1) se eu consigo montar uma máquina que sempre para com a resposta certa, o problema é computável, mesmo que seja lento; (2) se resolver o meu problema também resolveria o problema da parada, então ele é indecidível, porque o da parada já foi provado impossível. Obs.: só ver que o programa "está demorando" não prova nada, demora é sinal de problema difícil.

Orientações finais

1. Assista ao vídeo indicado.
2. Estude os conceitos apresentados.
3. Responda às questões propostas.
4. Realize a simulação da Máquina de Turing.
5. Faça os três testes solicitados.
6. Registre os resultados e as evidências da simulação.
7. Responda à reflexão sobre os limites computacionais.
8. Organize o material em um único arquivo.
9. Envie o arquivo conforme as orientações do professor.

Teoria da Computação — Atividade avaliativa | Profª Kadidja Valéria | Linguagens Formais e Autômatos |

Máquinas de Turing • Computabilidade • Limites Computacionais

Anexo A - Tabela de regras da minha máquina

Aluna: Julyane Ataides. Fiz a simulação num programa em Python que eu escrevi (maquina_de_turing.py). A ideia é marcar os símbolos, igual riscar item de uma lista: troco um 0 por A, vou até o primeiro 1 e troco por B, volto e repito. Se no final só sobrar A e B, tinha a mesma quantidade de 0 e 1.

Estados que eu usei

Estado	O que ele faz
inicio	procura o próximo 0 para marcar
procura_1	anda para a direita até achar o primeiro 1
volta	volta para a esquerda até achar o A
confere	acabaram os 0; confere se só tem B até o fim
aceita	deu certo!

Tabela de regras

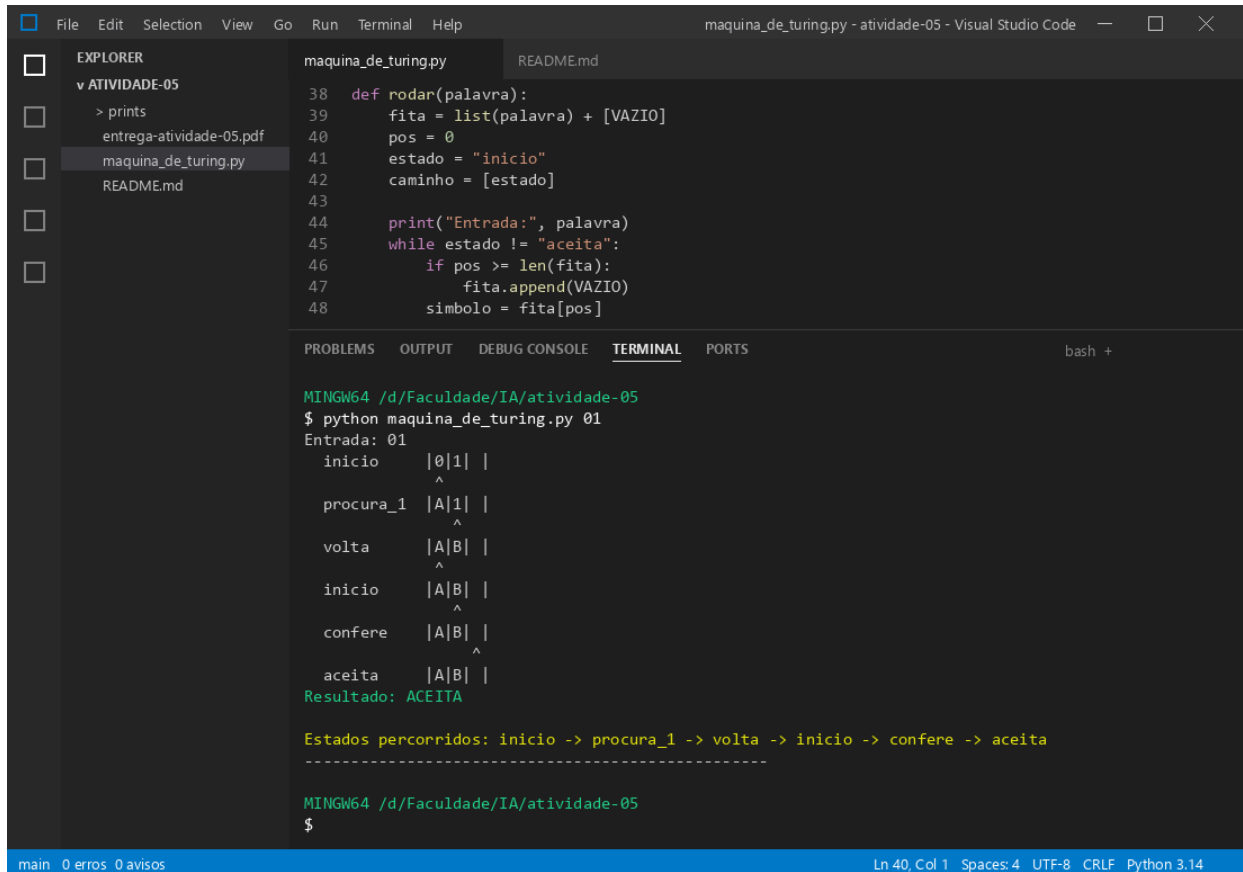
Estado	Lê	Escreve	Anda	Vai para
inicio	0	A	direita	procura_1
inicio	B	B	direita	confere
procura_1	0	0	direita	procura_1
procura_1	B	B	direita	procura_1
procura_1	1	B	esquerda	volta
volta	0	0	esquerda	volta
volta	B	B	esquerda	volta
volta	A	A	direita	inicio
confere	B	B	direita	confere
confere	vazio	vazio	direita	aceita

Se a máquina cair numa situação que não está na tabela, ela para e REJEITA. Por exemplo, confere lendo 1 quer dizer que sobrou um 1 sem par.

Também testei 0011 e 000111 (ACEITA) e 0, 1, 001 e 00111 (REJEITA). Todos deram o resultado esperado.

Prints da simulação (VS Code)

Print do teste 1 - entrada 01 (ACEITA)



The screenshot displays the Visual Studio Code interface. The Explorer sidebar on the left shows a project named 'ATIVIDADE-05' containing files 'prints', 'entrega-atividade-05.pdf', 'maquina_de_turing.py', and 'README.md'. The 'maquina_de_turing.py' file is open in the editor, showing a Python function 'rodar' that simulates a Turing machine. The function takes a word as input and processes it step by step, updating the state and the tape. The terminal at the bottom shows the execution of the script with the input '01'. The output displays the sequence of states and tape configurations for each step, culminating in the state 'aceita' (accepted). The final result is 'Resultado: ACEITA'. The terminal also shows the path to the script and the command used to run it.

```
maquina_de_turing.py  README.md

38 def rodar(palavra):
39     fita = list(palavra) + [VAZIO]
40     pos = 0
41     estado = "inicio"
42     caminho = [estado]
43
44     print("Entrada:", palavra)
45     while estado != "aceita":
46         if pos >= len(fita):
47             fita.append(VAZIO)
48             simbolo = fita[pos]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash +

```
MINGW64 /d/Faculdade/IA/atividade-05
$ python maquina_de_turing.py 01
Entrada: 01
  inicio   | 0|1| |
              ^
  procura_1 | A|1| |
              ^
  volta     | A|B| |
              ^
  inicio    | A|B| |
              ^
  confere   | A|B| |
              ^
  aceita    | A|B| |
Resultado: ACEITA

Estados percorridos: inicio -> procura_1 -> volta -> inicio -> confere -> aceita
-----

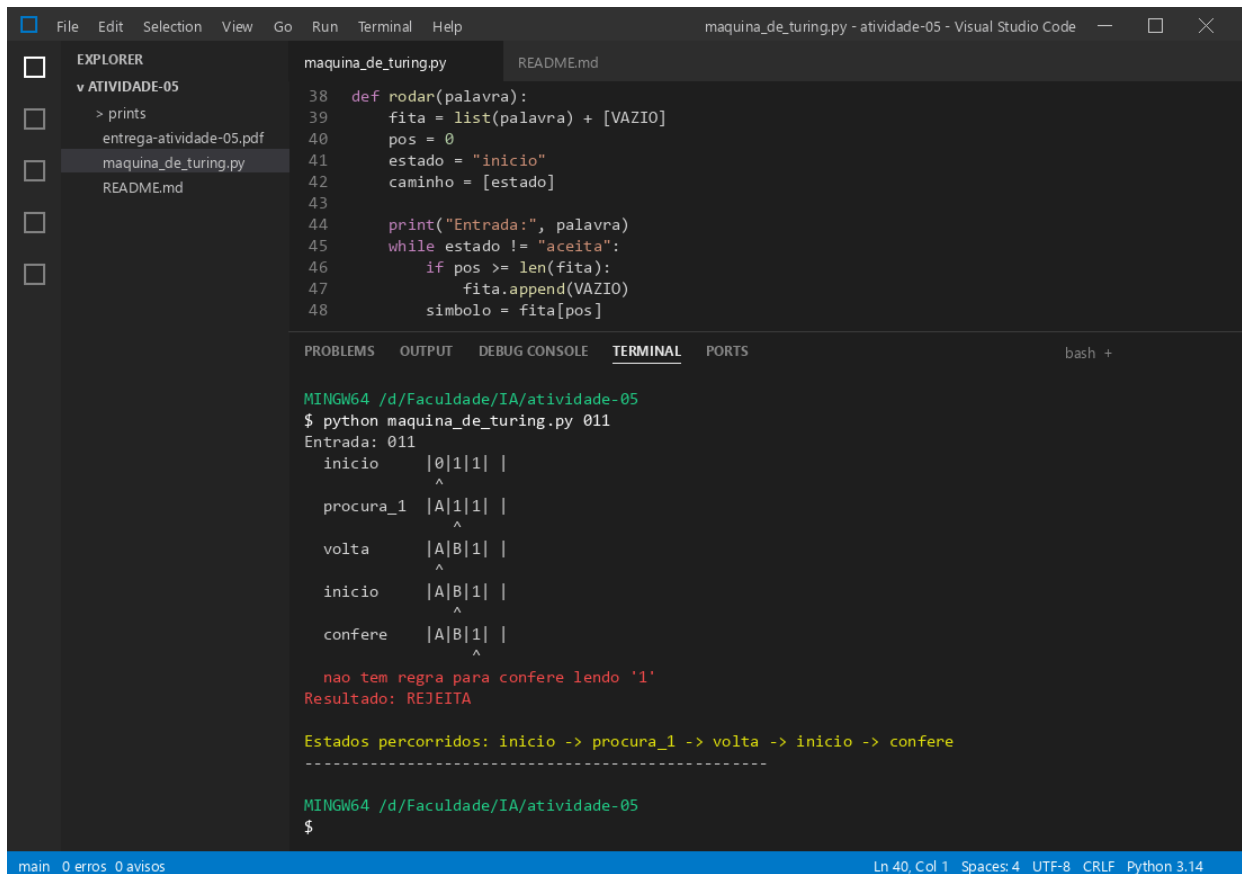
MINGW64 /d/Faculdade/IA/atividade-05
$
```

main 0 erros 0 avisos Ln 40, Col 1 Spaces: 4 UTF-8 CRLF Python 3.14

Print do teste 2 - entrada 00001111 (ACEITA)

[illegible]

Print do teste 3 - entrada 011 (REJEITA)



The image shows a Visual Studio Code window with the file `maquina_de_turing.py` open. The Explorer sidebar on the left shows the project structure under `ATIVIDADE-05`, including `prints`, `entrega-atividade-05.pdf`, `maquina_de_turing.py`, and `README.md`. The main editor displays the Python code for a Turing machine simulation. The terminal at the bottom shows the execution of the script with the input `011`. The output displays the sequence of states: `inicio`, `procura_1`, `volta`, `inicio`, and `confere`, with arrows indicating the transitions between them. The final result is `Resultado: REJEITA`. The status bar at the bottom indicates `main 0 erros 0 avisos` and the file encoding is `UTF-8`.

```
maquina_de_turing.py
38 def rodar(palavra):
39     fita = list(palavra) + [VAZIO]
40     pos = 0
41     estado = "inicio"
42     caminho = [estado]
43
44     print("Entrada:", palavra)
45     while estado != "aceita":
46         if pos >= len(fita):
47             fita.append(VAZIO)
48             simbolo = fita[pos]
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS bash +

```
MINGW64 /d/Faculdade/IA/atividade-05
$ python maquina_de_turing.py 011
Entrada: 011
inico      |0|1|1| |
            ^
procura_1  |A|1|1| |
            ^
volta      |A|B|1| |
            ^
inico      |A|B|1| |
            ^
confere    |A|B|1| |
            ^
    nao tem regra para confere lendo '1'
Resultado: REJEITA

Estados percorridos: inicio -> procura_1 -> volta -> inicio -> confere
-----

MINGW64 /d/Faculdade/IA/atividade-05
$
```

main 0 erros 0 avisos Ln 40, Col 1 Spaces: 4 UTF-8 CRLF Python 3.14